EXP 3	**Implement a Generative Adversarial Network (GAN) to generate synthetic images using the MNIST dataset.**

AIM:

To implement a **Generative Adversarial Network (GAN)** that learns to generate realistic synthetic handwritten digit images by training on the **MNIST dataset**. The GAN consists of a **Generator** that produces fake images from random noise and a **Discriminator** that distinguishes real images from fake ones. Through adversarial training, the generator learns to produce increasingly realistic images.

PROCEDURE:

1. **Load Dataset:** Load the MNIST dataset using TensorFlow/Keras. Normalize pixel values from [0, 255] to [-1, 1] and reshape images to include a channel dimension.
2. **Prepare Dataset:** Create a TensorFlow data pipeline with shuffling (buffer size 60,000) and batching (batch size 256) for efficient training.
3. **Build Generator:** Design a Sequential model that takes a 100-dimensional noise vector as input, passes through Dense layers (256, 512, 784 units) with ReLU activation and Batch Normalization, and reshapes output to a 28x28x1 image using Tanh activation.
4. **Build Discriminator:** Design a Sequential model that flattens the 28x28x1 input image, passes through Dense layers (512, 256 units) with ReLU activation, and outputs a single sigmoid value representing real or fake probability.
5. **Compile Discriminator:** Compile the discriminator with Adam optimizer and binary cross-entropy loss for binary classification.
6. **Build GAN Model:** Freeze the discriminator weights and connect the generator and discriminator into a combined GAN model. Compile the GAN with Adam optimizer and binary cross-entropy loss.
7. **Train GAN:** For each epoch and each batch: generate fake images using random noise, train the discriminator on real and fake images separately, then train the generator via the GAN model to fool the discriminator.
8. **Generate and Visualize Images:** After training for 50 epochs, generate 10 synthetic images from random noise using the trained generator and display them using Matplotlib.

CODE:

```
# ==========================================
# GAN Implementation for MNIST Image Generation
# ==========================================

import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------
# Load MNIST Dataset
# ------------------------------------------
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize images to [-1,1]
x_train = (x_train.astype('float32') - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

BUFFER_SIZE = 60000
BATCH_SIZE  = 256

dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

```
# ----------------------------------------
# Generator Model
# ----------------------------------------
def build_generator():
    model = tf.keras.Sequential()
    model.add(layers.Dense(256, activation="relu", input_dim=100))
    model.add(layers.BatchNormalization())
    model.add(layers.Dense(512, activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.Dense(784, activation="tanh"))
    model.add(layers.Reshape((28, 28, 1)))
    return model

# ----------------------------------------
# Discriminator Model
# ----------------------------------------
def build_discriminator():
    model = tf.keras.Sequential()
    model.add(layers.Flatten(input_shape=(28, 28, 1)))
    model.add(layers.Dense(512, activation="relu"))
    model.add(layers.Dense(256, activation="relu"))
    model.add(layers.Dense(1, activation="sigmoid"))
    return model

generator     = build_generator()
discriminator = build_discriminator()

# ----------------------------------------
# Compile Discriminator
# ----------------------------------------
discriminator.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# ----------------------------------------
# GAN Model
# ----------------------------------------
discriminator.trainable = False

gan_input  = tf.keras.Input(shape=(100,))
fake_image = generator(gan_input)
gan_output = discriminator(fake_image)

gan = tf.keras.Model(gan_input, gan_output)
gan.compile(optimizer="adam", loss="binary_crossentropy")

# ----------------------------------------
# Training GAN
# ----------------------------------------
EPOCHS    = 50
noise_dim = 100

for epoch in range(EPOCHS):
    for real_images in dataset:
        batch_size  = real_images.shape[0]
        noise       = np.random.normal(0, 1, (batch_size, noise_dim))
        fake_images = generator.predict(noise, verbose=0)
        real_labels = np.ones((batch_size, 1))
        fake_labels = np.zeros((batch_size, 1))
        d_loss_real = discriminator.train_on_batch(real_images, real_labels)
        d_loss_fake = discriminator.train_on_batch(fake_images, fake_labels)
        noise       = np.random.normal(0, 1, (batch_size, noise_dim))
        g_loss      = gan.train_on_batch(noise, real_labels)
    print(f"Epoch {epoch+1}/{EPOCHS} completed")

# ----------------------------------------
# Generate Synthetic Images
```

```
# ------------------------------------------
noise            = np.random.normal(0, 1, (10, 100))
generated_images = generator.predict(noise)
generated_images = 0.5 * generated_images + 0.5

plt.figure(figsize=(10, 3))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(generated_images[i].reshape(28, 28), cmap='gray')
    plt.axis("off")
plt.show()
```

Output:

The GAN was trained for 50 epochs on the MNIST dataset. The following 10 synthetic handwritten digit images were generated by the trained Generator model:

Result:

The Generative Adversarial Network (GAN) was successfully implemented and trained on the **MNIST dataset** for 50 epochs. The **Generator** learned to produce synthetic handwritten digit images from random noise vectors, while the **Discriminator** learned to distinguish real MNIST images from generated ones.

The generated images demonstrate that the GAN successfully captured the visual patterns of handwritten digits. The adversarial training process enabled the generator to progressively improve image quality, showcasing the effectiveness of GANs in **synthetic image generation** and **data augmentation** tasks.